

Analyzing Complexity Class $\mathbf{P}$ through a Quantum Lens: Fine-Grained Bounds and APSP

Rupanshu Shah
Indian Institute of Technology Delhi

September 30, 2026

Abstract

We study the fine-grained complexity of the All-Pairs Shortest Paths problem (APSP) through its subcubic equivalence with Negative-Weight Triangle detection (NWT). This part fixes the problem definitions, the input/output model, and a uniform way of accounting for reductions (number of oracle calls, adaptivity, oracle-instance parameters, overhead). We then give a self-contained proof of the reduction chain $\text{APSP} \rightarrow \text{MPMM} \rightarrow \text{APNWT} \rightarrow \text{NWT-Find} \rightarrow \text{NWT-Det}$, tracking how the parameters (n, W) (size, maximum absolute weight) evolve along it. Known classical upper and lower bounds and the quantum analysis are treated in later sections.

Contents

1	Introduction	3
2	Preliminaries	3
2.1	Notation and model of computation	3
2.2	Reductions and their profiles	4
3	Problems	4
3.1	All-Pairs Shortest Paths	4
3.2	Negative-Weight Triangle	6
3.3	Min-Plus Matrix Multiplication	7
3.4	All-Pairs Negative-Weight Triangle	7
4	The Reductions	7
4.1	NWT: finding reduces to detection	7
4.2	NWT reduces to APSP	9
4.3	APSP and MPMM are equivalent	9
4.4	The key reduction: MPMM reduces to APNWT-Det	10
4.5	APNWT: finding reduces to detection	12
4.6	APNWT reduces to NWT-Find	13
5	Main Result: from APSP to NWT and back	14
A	Deferred Proofs	16
A.1	Proof: orientation	16
A.2	Proof: tripartite vs. general	16
A.3	Structure of walks and shortest paths	17
A.4	Proof: NWT to APSP	17
A.5	Proof: APSP to MPMM	18

A.6	Proof: MPMM to APSP	18
A.7	Proof: APNWT finding to MPMM	18
A.8	Proof: APNWT to NWT-Find	19

1 Introduction

APSP on n -vertex graphs is solvable in $O(n^3)$ time, and despite decades of work no algorithm running in $O(n^{3-\delta})$ time for a constant $\delta > 0$ is known; the best known bound is $n^3/2^{\Omega(\sqrt{\log n})}$ [Wil14]. Vassilevska Williams and Williams [VWW18] showed that this question is equivalent to the same question for a far simpler-looking *decision* problem: does an edge-weighted graph contain a triangle of negative total weight? The surprising direction is that an algorithm returning one bit (NWT) can be used to compute n^2 distances (APSP) while preserving truly subcubic running time.

This write-up gives a self-contained account of that equivalence. We fix definitions in [Section 3](#), prove the individual reductions in [Section 4](#), and compose them into the main result in [Section 5](#):

$$\text{APSP} \longrightarrow \text{MPMM} \longrightarrow \text{APNWT} \longrightarrow \text{NWT-Find} \longrightarrow \text{NWT-Det} \longrightarrow \text{APSP}.$$

For every arrow we record four characteristics: the number of oracle calls, whether calls are adaptive, the parameters of the oracle instances, and the overhead time ([Definition 2.2](#)).

Convention. Proofs are deferred to [Section A](#) unless stated otherwise. The proofs of [Lemma 4.2](#) (finding from detection) and [Theorem 4.10](#) (the key reduction) are given in the main text, as is the composition argument of [Section 5](#).

2 Preliminaries

2.1 Notation and model of computation

We write $[n] = \{1, \dots, n\}$, $\sqcup$ for disjoint union, and $\log$ for $\log_2$. For an integer $W \geq 1$ let

$$\mathbb{Z}_W := \{-W, \dots, W\}, \quad \mathbb{Z}_W^\infty := \mathbb{Z}_W \cup \{+\infty\}.$$

We use the conventions $x + \infty = \infty$ for every $x \in \mathbb{Z} \cup \{\infty\}$, $\infty > x$ for every $x \in \mathbb{Z}$, and $\min \emptyset = +\infty$. In every weight matrix, the entry $+\infty$ encodes an *absent edge*. The value $-\infty$ never occurs in an input; it only occurs in the output of APSP.

Model. Classical algorithms run on a word RAM with words of $\Theta(\log n + \log W)$ bits. All integers manipulated in this write-up have absolute value at most $\text{poly}(n) \cdot W$, so each fits in $O(1)$ words, and addition, comparison, multiplication by n , and floor division by n take $O(1)$ time. The sentinels $\pm\infty$ are encoded by reserved words. Inputs are matrices stored in random-access memory; reading one entry costs $O(1)$. (The quantum query model, in which the input is accessed through an oracle, is introduced in a later section.)

Running-time functions. Every problem Π below has two parameters: a *size* n and a *weight bound* W (all input weights lie in $\mathbb{Z}_W$). $T_\Pi(n, W)$ denotes the worst-case running time of the algorithm under consideration over all instances with these parameters. T_Π is nondecreasing in W by definition (an instance with weights in $\mathbb{Z}_{W'}$, $W' \leq W$, is an instance with weights in $\mathbb{Z}_W$), and, up to an additive $O(n^2)$, nondecreasing in n by the following padding convention.

Remark 2.1 (Padding). An instance of size $n' < n$ is turned into one of size n by adding vertices whose incident weight entries are all $+\infty$ (for matrices: adding rows and columns of $+\infty$). Padded vertices lie on no finite path and in no triangle, so the answer is unchanged (and is read off in $O(n^2)$ time). Hence instances whose parts have *at most* n vertices may always be treated as instances of size exactly n .

2.2 Reductions and their profiles

Definition 2.2 (Reduction and its profile). Let Π_1, Π_2 be computational problems parametrized by (n, W) . A *reduction* from Π_1 to Π_2 , written $\Pi_1 \leq \Pi_2$, is a *Turing reduction*: an algorithm that solves every instance of Π_1 while querying an oracle for Π_2 ; each query is an instance of Π_2 , answered in unit time, and the answers may be used arbitrarily. Every reduction is described by its *profile*:

- (R1) **Calls**: the worst-case number of oracle calls.
- (R2) **Adaptivity**: *non-adaptive* if all queries are determined by the input alone (so they can be issued as one batch); *adaptive* if some query depends on answers to earlier ones. For adaptive reductions we also state the number of sequential rounds.
- (R3) **Oracle instances**: the parameters (n_i, W_i) of every query, as functions of (n, W) .
- (R4) **Overhead**: the time spent outside oracle calls, including writing queries and reading answers.

Replacing the oracle by an algorithm for Π_2 yields the bound

$$T_{\Pi_1}(n, W) \leq \text{Overhead}(n, W) + \sum_i T_{\Pi_2}(n_i, W_i). \quad (1)$$

Shorthand: $\Pi_1 \leq_f \Pi_2$ denotes a reduction with one non-adaptive call on an instance with parameters $(O(n), W)$ and overhead $O(f(n))$.

Remark 2.3 (Composition). If $\Pi_1 \leq \Pi_2$ makes c_1 calls with overhead o_1 , and every query to Π_2 is answered by a reduction $\Pi_2 \leq \Pi_3$ making c_2 calls with overhead o_2 (both evaluated at that query's parameters), then $\Pi_1 \leq \Pi_3$ with at most $c_1 c_2$ calls, overhead at most $o_1 + c_1 o_2$, oracle-instance parameters obtained by composing the parameter maps, and it is adaptive if either reduction is.

Remark 2.4 (Explicit vs. implicit sub-instances). Several reductions query *induced sub-instances* of the input. We charge the time to write them down explicitly. Alternatively, a sub-instance can be passed implicitly (as index maps into the original input), which costs $O(1)$ per access; this is the relevant view in the query model used later.

Definition 2.5 (Truly subcubic; subcubic equivalence). A problem Π is *truly subcubic* if there are constants $\delta > 0$ and $d \geq 0$ and an algorithm with $T_{\Pi}(n, W) = O(n^{3-\delta} \log^d(W+1))$ for all $n, W \geq 1$. Problems Π_1, Π_2 are *subcubic equivalent*, $\Pi_1 \equiv_3 \Pi_2$, if Π_1 is truly subcubic if and only if Π_2 is.

This is the notion of [VWW18] specialised to our parameters; there the size measure is refined to include $\log W$ and a reduction is required to preserve truly subcubic running times for every δ . We prove the needed implications directly from the profiles.

3 Problems

3.1 All-Pairs Shortest Paths

Let $G = (V, E, w)$ be a directed graph with $|V| = n$ and weights $w : E \rightarrow \mathcal{D}$ for a domain $\mathcal{D} \subseteq \mathbb{R}$. A *walk* from u to v is a sequence $p = (v_0 = u, v_1, \dots, v_\ell = v)$ with $(v_{i-1}, v_i) \in E$; its *length* is $\ell \geq 0$ (edges counted with multiplicity) and its *weight* is $w(p) = \sum_{i=1}^{\ell} w(v_{i-1}, v_i)$. A *path* is a walk with distinct vertices; a *cycle* is a closed walk $v_0 = v_\ell$ with $\ell \geq 1$ whose vertices $v_0, \dots, v_{\ell-1}$ are distinct. The *distance* is

$$\delta(u, v) := \inf\{w(p) : p \text{ a walk from } u \text{ to } v\} \in \mathbb{R} \cup \{\pm\infty\}.$$

We write $u \rightsquigarrow v$ if some walk from u to v exists (in particular $u \rightsquigarrow u$).

Definition 3.1 (All-Pairs Shortest Paths, APSP). **Input:** a directed graph on V , $|V| = n$, given as its weight matrix $W_G \in (\mathcal{D} \cup \{+\infty\})^{V \times V}$, where $W_G[u, v] = w(u, v)$ if $(u, v) \in E$ and $W_G[u, v] = +\infty$ otherwise.

Output: the matrix $D \in (\mathcal{D}' \cup \{\pm\infty\})^{V \times V}$ with $D[u, v] = \delta(u, v)$ for all $u, v \in V$, where $\mathcal{D}'$ is the set of finite sums of elements of $\mathcal{D}$.

The three kinds of output entries mean the following (Lemma A.1):

Remark 3.2. (i) If there is no walk from u to v , then $\delta(u, v) = +\infty$. (ii) $\delta(u, v) = -\infty$ if and only if some walk from u to v visits a vertex that lies on a negative-weight cycle. (iii) Otherwise $\delta(u, v)$ is finite and is attained by a *path*; in particular it is the weight of a walk with at most $n - 1$ edges.

Remark 3.3 (Self-loops). There are no self-loops (unless stated otherwise): $W_G[u, u] = +\infty$. A non-negative self-loop never decreases a distance and can be dropped; a negative self-loop at z is a negative cycle, and the reduction of Theorem 4.6 handles it by adding z to the set S defined there.

Variants. The weight domain $\mathcal{D}$ determines the variant:

- (a) *Real weights*, $\mathcal{D} = \mathbb{R}$, on a real RAM (unit-cost addition and comparison of reals).
- (b) *Unbounded integer weights*, $\mathcal{D} = \mathbb{Z}$; the input length then depends on the bit-lengths of the weights.
- (c) *b -bit integer weights*, $\mathcal{D} = \mathbb{Z}_W$ with $W = 2^b - 1$, where $b = b(n)$ may be any function of n . We denote this variant by $\text{APSP}(n, W)$.

Further variants restrict the graph (undirected graphs, non-negative weights); undirected APSP with non-negative weights is subcubic equivalent to directed APSP [VWW18, Thm. 5.1]. We work with directed graphs throughout.

Assumption 3.4 (Standing weight regime). Unless stated otherwise, APSP means $\text{APSP}(n, W)$ with $b = \Theta(\log n)$ -bit integer weights, i.e. $W = n^{\Theta(1)}$; in particular $W \leq n^c$ for a constant c . Every weight, and every finite distance, fits in $O(1)$ machine words.

Remark 3.5 (Why this regime). (i) *Why integers.* The reduction from MPMM to APNWT (Theorem 4.10) is a binary search over output *values* and terminates because values are integers; its number of calls is $O(\log W)$. For real weights such value-based searches are unavailable and one needs comparison-based techniques (cf. [VWW18, Sec. 4.3]).

- (ii) *Why not more than $O(\log n)$ bits.* With $b = \omega(\log n)$ the input occupies $\Theta(n^2 b)$ bits, and adding or comparing two weights costs $\Theta(b)$ bit operations. Even Floyd–Warshall then costs $\Theta(n^3 b)$ bit operations, and the extra factor measures the length of the numbers, not the combinatorial difficulty of the problem. (Our word model, with words of $\Theta(\log n + \log W)$ bits, hides this cost; that is harmless only when $\log W = O(\log n)$.) Allowing larger weights thus *artificially bloats* the problem without changing its structure.
- (iii) *Why not fewer bits.* Small weights make the problem easier: the min-plus product of matrices with entries in $\mathbb{Z}_W$ can be computed in $\tilde{O}(W n^\omega)$ time [AGM97], which is truly subcubic when $W \leq n^{3-\omega-\varepsilon}$. Small weights are in any case covered by all results below, since they use only the upper bound $W \leq n^c$.
- (iv) *Stability.* Every reduction in this write-up maps instances with $O(\log n)$ -bit weights to oracle instances with $O(\log n')$ -bit weights, where n' is the size of the oracle instance (Remark 5.1).

Remark 3.6 (Output size). By Remark 3.2(iii), every finite output entry lies in $\mathbb{Z}_{(n-1)W}$.

3.2 Negative-Weight Triangle

Definition 3.7 (Tripartite instance). A *tripartite instance with parameters* (n, W) consists of a vertex set $V = A \sqcup B \sqcup C$ with $|A| = |B| = |C| = n$ and three weight matrices with entries in $\mathbb{Z}_W^\infty$:

$$W_{AB} \in (\mathbb{Z}_W^\infty)^{A \times B}, \quad W_{BC} \in (\mathbb{Z}_W^\infty)^{B \times C},$$

and either $W_{AC} \in (\mathbb{Z}_W^\infty)^{A \times C}$ (transitive case) or $W_{CA} \in (\mathbb{Z}_W^\infty)^{C \times A}$ (circular case). They encode a directed graph whose edges go from A to B , from B to C , and either from A to C or from C to A ; the entry $+\infty$ marks an absent edge. The *size* of the instance is the part size n (the graph has $3n$ vertices).

Definition 3.8 (Negative-weight triangles). Let $(a, b, c) \in A \times B \times C$.

- **Transitive:** (a, b, c) is a *transitive negative-weight triangle* if

$$W_{AB}[a, b] + W_{BC}[b, c] + W_{AC}[a, c] < 0.$$

It consists of the path $a \rightarrow b \rightarrow c$ and the “shortcut” $a \rightarrow c$.

- **Circular:** (a, b, c) is a *circular negative-weight triangle* if

$$W_{AB}[a, b] + W_{BC}[b, c] + W_{CA}[c, a] < 0.$$

It is the directed 3-cycle $a \rightarrow b \rightarrow c \rightarrow a$.

Since $+\infty$ absorbs addition, a negative sum forces all three edges to be present; the three vertices are distinct since they lie in different parts.

Definition 3.9 (NWT detection and finding). Given a tripartite instance with parameters (n, W) :

- **Detection** (NWT-Det(n, W)): output YES if a negative-weight triangle exists and NO otherwise.
- **Finding** (NWT-Find(n, W)): output some negative-weight triangle (a, b, c) , or $\perp$ if none exists.

Each comes in a transitive and a circular version.

Definition 3.10 (General (non-tripartite) NWT). A *general instance with parameters* (n, W) is a directed graph on V , $|V| = n$, given by $W_G \in (\mathbb{Z}_W^\infty)^{V \times V}$ with $W_G[u, u] = +\infty$. A triple $(u, v, x) \in V^3$ is a *transitive* negative-weight triangle if $W_G[u, v] + W_G[v, x] + W_G[u, x] < 0$, and a *circular* one if $W_G[u, v] + W_G[v, x] + W_G[x, u] < 0$. (Such u, v, x are automatically distinct: any coincidence puts a diagonal entry $+\infty$ into the sum.) Detection and finding are defined as in [Definition 3.9](#).

The variants are equivalent up to $O(n^2)$ overhead; each reduction below makes one non-adaptive call, does not change W , and works for both detection and finding.

Proposition 3.11 (Orientation). *For tripartite instances,*

$$\begin{aligned} \text{Circular NWT}(n, W) &\leq_{n^2} \text{Transitive NWT}(n, W), \\ \text{Transitive NWT}(n, W) &\leq_{n^2} \text{Circular NWT}(n, W). \end{aligned}$$

Proposition 3.12 (Tripartite vs. general). *For either orientation,*

$$\begin{aligned} \text{Tripartite NWT}(n, W) &\leq_{n^2} \text{General NWT}(3n, W), \\ \text{General NWT}(n, W) &\leq_{n^2} \text{Tripartite NWT}(n, W). \end{aligned}$$

Remark 3.13 (Undirected graphs). An undirected graph is a general instance with a symmetric matrix W_G . A triangle $\{u, v, x\}$ has negative total weight if and only if (u, v, x) is a transitive negative-weight triangle of W_G , so undirected NWT is a special case of general transitive NWT.

Remark 3.14 (Convention). Henceforth NWT refers to the *transitive, tripartite* version. It matches the min-plus product exactly: (a, b, c) is a negative triangle iff $W_{AB}[a, b] + W_{BC}[b, c] < -W_{AC}[a, c]$, i.e. a two-step path $a \rightarrow b \rightarrow c$ is shorter than the negated shortcut (see [Remark 3.20](#)). The *size* of an NWT instance is its part size n .

Observation 3.15. $\text{NWT-Det}(n, W) \leq \text{NWT-Find}(n, W)$ with one non-adaptive call on the same instance and $O(1)$ overhead: answer YES iff the finder does not return $\perp$.

The converse reduction is [Lemma 4.2](#).

3.3 Min-Plus Matrix Multiplication

Definition 3.16 (Min-plus product, MPMM). **Input:** matrices $M, N \in (\mathbb{Z}_W^\infty)^{n \times n}$.

Output: $P = M * N \in (\mathbb{Z}_{2W}^\infty)^{n \times n}$, where

$$P[i, j] := \min_{k \in [n]} (M[i, k] + N[k, j]).$$

We write $\text{MPMM}(n, W)$ for this problem.

Remark 3.17. $*$ is matrix multiplication over the $(\min, +)$ -semiring (“distance product”), with $+\infty$ as the additive and 0 as the multiplicative identity; in particular $*$ is associative. The closely related *verification* problem (given M, N, P , decide whether $P = M * N$) is also subcubic equivalent to APSP [[VWW18](#), Thm. 1.1]; we do not need it here.

3.4 All-Pairs Negative-Weight Triangle

Definition 3.18 (All-pairs NWT, APNWT). Let (W_{AB}, W_{BC}, W_{AC}) be a (transitive) tripartite instance with parameters (n, W) . For $(a, c) \in A \times C$, a *witness* for (a, c) is a vertex $b \in B$ such that (a, b, c) is a negative-weight triangle.

- **Detection** ($\text{APNWT-Det}(n, W)$): output $Y \in \{0, 1\}^{A \times C}$ with $Y[a, c] = 1$ iff (a, c) has a witness.
- **Finding** ($\text{APNWT-Find}(n, W)$): output $F \in (B \cup \{\perp\})^{A \times C}$, where $F[a, c]$ is a witness for (a, c) if one exists and $F[a, c] = \perp$ otherwise.

Observation 3.19. Each of the following makes one non-adaptive call on the same instance with $O(n^2)$ overhead: $\text{NWT-Det} \leq \text{APNWT-Det}$ (answer YES iff some $Y[a, c] = 1$), and $\text{APNWT-Det} \leq \text{APNWT-Find}$ (set $Y[a, c] = 1$ iff $F[a, c] \neq \perp$).

Remark 3.20 (APNWT is a thresholded min-plus product). Since $W_{AB}[a, b] + W_{BC}[b, c] + W_{AC}[a, c] < 0$ for some b iff $\min_b (W_{AB}[a, b] + W_{BC}[b, c]) < -W_{AC}[a, c]$,

$$Y[a, c] = 1 \iff (W_{AB} * W_{BC})[a, c] < -W_{AC}[a, c].$$

So APNWT-Det compares a min-plus product entrywise against a threshold matrix. This is the bridge exploited in [Theorem 4.10](#).

4 The Reductions

4.1 NWT: finding reduces to detection

The reduction needs a mild regularity condition on the running time.

Assumption 4.1 (Regularity). A function $T : \mathbb{N} \rightarrow \mathbb{R}_{>0}$ is *regular* if $T(m)/m$ is nondecreasing and $T(m) \geq m^2$ for all $m \geq 1$.

This costs no generality classically. Any classical NWT-Det algorithm reads $\Omega(m^2)$ entries in the worst case: with $W_{AB} \equiv W_{BC} \equiv 0$ and $W_{AC} \equiv 0$ except possibly one entry equal to -1 , every entry of W_{AC} must be read. If T bounds the running time, so does $\max\{T(m), m^2\}$, and $\max\{T(m), m^2\}/m = \max\{T(m)/m, m\}$ is nondecreasing whenever $T(m)/m$ is. Every bound of the form $Km^\alpha \log^\beta(m+1)$ with $\alpha \geq 2$, $\beta \geq 0$, $K \geq 1$ is regular.

Lemma 4.2 (Finding from detection; cf. [VWW18, Lem. 4.1]). *If $\text{NWT-Det}(m, W)$ can be solved in time $T(m)$ for every $m \geq 1$, for a regular T , then $\text{NWT-Find}(n, W)$ can be solved in time $16T(n) + O(n^2) = O(T(n))$.*

This is a Turing reduction with the following profile (the weight bound W is fixed throughout).

Calls	At most $8\lceil \log n \rceil$ calls to NWT-Det.
Adaptivity	Adaptive, with $\lceil \log n \rceil$ rounds. The 8 calls of a round are mutually independent (a non-adaptive batch), but the instance split in the next round depends on their answers.
Oracle instances	In round $i \in [\lceil \log n \rceil]$: 8 instances $\text{NWT-Det}(\lceil n/2^i \rceil, W)$, which are induced sub-instances; the size halves every round and the weights are unchanged.
Overhead	$O(n^2)$ to write the sub-instances explicitly (a geometric sum), or $O(\log n)$ with implicit sub-instances (Remark 2.4); plus $O(1)$ for the base case.

Proof. Algorithm. Let $\mathcal{I} = (A, B, C)$ be the current instance, of size m (initially $\mathcal{I} = G$, $m = n$). If $m = 1$, check the unique triple in $O(1)$ time and return it if it is negative, and $\perp$ otherwise. If $m \geq 2$, split each of A, B, C into two halves of sizes $\lceil m/2 \rceil$ and $\lfloor m/2 \rfloor$, say $A = A_1 \sqcup A_2$, $B = B_1 \sqcup B_2$, $C = C_1 \sqcup C_2$, and pad the halves to size $\lceil m/2 \rceil$ (Remark 2.1). Run NWT-Det on the eight induced sub-instances $\mathcal{I}_{xyz} = (A_x, B_y, C_z)$, $x, y, z \in \{1, 2\}$. If all eight answer NO, return $\perp$. Otherwise pick any $\mathcal{I}_{xyz}$ that answered YES and recurse on it.

Correctness. Every triple $(a, b, c) \in A \times B \times C$ lies in exactly one of the eight sub-instances, and has the same weight there. Hence $\mathcal{I}$ contains a negative triangle iff some $\mathcal{I}_{xyz}$ does. Consequently: if all eight answers are NO, then $\mathcal{I}$ has no negative triangle; and whenever the algorithm recurses, the new instance contains a negative triangle. By induction the current instance always contains a negative triangle after the first level, so the base case, whose instance consists of a single triple, finds it. A triangle found in a sub-instance is a negative triangle of G , because sub-instances are induced and padded vertices lie in no triangle.

Running time. Writing $T'(m)$ for the time of the finder on size m ,

$$T'(m) \leq 8T(\lceil m/2 \rceil) + T'(\lceil m/2 \rceil) + O(m^2), \quad T'(1) = O(1).$$

Let $n_0 = n$ and $n_i = \lceil n_{i-1}/2 \rceil$; then $n_i = \lceil n/2^i \rceil \leq n/2^i + 1$, and $n_D = 1$ for $D = \lceil \log n \rceil$. Unrolling,

$$T'(n) \leq \sum_{i=1}^D 8T(n_i) + O\left(\sum_{i=0}^{D-1} n_i^2\right) + O(1).$$

Since $T(m)/m$ is nondecreasing and $n_i \leq n$, we have $T(n_i) \leq \frac{n_i}{n}T(n)$. Moreover $\sum_{i=1}^D n_i \leq \sum_{i \geq 1} n/2^i + D \leq n + D \leq 2n$, because $D = \lceil \log n \rceil \leq n$. Hence $\sum_{i=1}^D 8T(n_i) \leq \frac{8T(n)}{n} \cdot 2n = 16T(n)$. Using $(x+1)^2 \leq 2x^2 + 2$, we get $\sum_{i=0}^{D-1} n_i^2 \leq \sum_{i \geq 0} 2n^2/4^i + 2D = O(n^2)$. Therefore $T'(n) \leq 16T(n) + O(n^2) = O(T(n))$, where the last step uses $T(n) \geq n^2$. The number of calls is $8D = 8\lceil \log n \rceil$. $\square$

Remark 4.3. By [Observation 3.15](#) and [Lemma 4.2](#), detection and finding have the same complexity up to a constant factor, on instances with the same parameters. We therefore write NWT for either problem when the distinction does not matter.

4.2 NWT reduces to APSP

Theorem 4.4. $NWT\text{-}Det(n, W) \leq_{n^2} APSP(3n, W)$.

Construction. Given (W_{AB}, W_{BC}, W_{AC}) , build the directed graph H on $A \sqcup B \sqcup C$ with only the edges $a \rightarrow b$ of weight $W_{AB}[a, b]$ and $b \rightarrow c$ of weight $W_{BC}[b, c]$ (entries $+\infty$ give no edge). The A – C edges are *not* put into H . Compute $D = APSP(H)$ and answer YES iff

$$\exists (a, c) \in A \times C : \quad D[a, c] + W_{AC}[a, c] < 0.$$

H is a DAG with layers $A \rightarrow B \rightarrow C$, so it has no negative cycles, and $D[a, c] = \min_b (W_{AB}[a, b] + W_{BC}[b, c])$.

Calls	1 call to APSP.
Adaptivity	Non-adaptive.
Oracle instances	$APSP(3n, W)$: $3n$ vertices, weights unchanged.
Overhead	$O(n^2)$: build H , then scan all (a, c) .

Remark 4.5. (i) The same construction solves APNWT-Det: set $Y[a, c] = 1$ iff $D[a, c] + W_{AC}[a, c] < 0$. Hence $APNWT\text{-}Det(n, W) \leq_{n^2} APSP(3n, W)$.

(ii) Adding the A – C information to H as reversed edges $c \rightarrow a$ would be wrong: H would then contain cycles $a \rightarrow b \rightarrow c \rightarrow a' \rightarrow b' \rightarrow c' \rightarrow a$ whose weight is not a sum of triangle weights, so negative cycles, and hence entries $-\infty$, could appear without any negative triangle.

4.3 APSP and MPMM are equivalent

Theorem 4.6 (APSP reduces to MPMM). $APSP(n, W) \leq MPMM$ with the profile below. Consequently $T_{APSP}(n, W) \leq (\lceil \log n \rceil + 1) T_{MPMM}(n, nW) + O(n^2 \log n)$.

Construction (repeated squaring, then negative-cycle post-processing). Let $t := \lceil \log n \rceil$.

1. Let $D_0 \in (\mathbb{Z}_W^\infty)^{V \times V}$ with $D_0[u, u] = 0$ and $D_0[u, v] = W_G[u, v]$ for $u \neq v$.
2. For $s = 1, \dots, t$: $D_s := D_{s-1} * D_{s-1}$ (one MPMM call each). Let $D := D_t$.
3. Let $S := \{z \in V : D[z, z] < 0\}$. Every vertex on a negative cycle lies in S , and every vertex of S reaches, and is reached from, a negative cycle ([Section A](#)). Define $X, Y \in \{0, +\infty\}^{V \times V}$ by

$$X[u, z] = \begin{cases} 0, & z \in S \text{ and } D[u, z] < \infty \\ +\infty, & \text{otherwise} \end{cases} \quad Y[z, v] = \begin{cases} 0, & z \in S \text{ and } D[z, v] < \infty \\ +\infty, & \text{otherwise,} \end{cases}$$

and compute $Z := X * Y$ (one more call).

4. Output $\delta(u, v) = -\infty$ if $Z[u, v] = 0$, and $\delta(u, v) = D[u, v]$ otherwise.

Here $D_s[u, v]$ is the minimum weight of a walk from u to v with at most 2^s edges ([Lemma A.2](#)). Since $2^t \geq n$, the matrix D captures every path and every cycle. Moreover $Z[u, v] = 0$ iff $u \rightsquigarrow z \rightsquigarrow v$ for some $z \in S$.

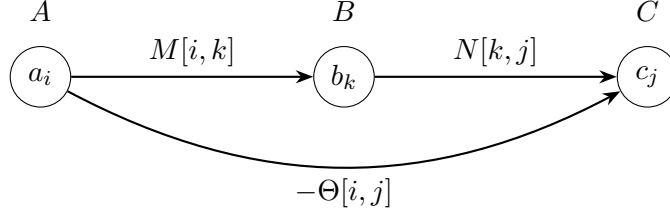


Figure 1: The instance G_Θ used in [Theorem 4.10](#). The pair (a_i, c_j) has a witness iff $P[i, j] = \min_k (M[i, k] + N[k, j]) < \Theta[i, j]$.

Calls	$\lceil \log n \rceil + 1$ calls to MPMM.
Adaptivity	Adaptive, with $\lceil \log n \rceil + 1$ sequential rounds of one call: each squaring uses the previous product, and the last call depends on D .
Oracle instances	The s -th squaring is an instance of $\text{MPMM}(n, 2^{s-1}W)$, since a walk with at most 2^{s-1} edges has weight in $\mathbb{Z}_{2^{s-1}W}$. As $2^{t-1} < n$, all squarings are instances of $\text{MPMM}(n, nW)$. The final call is $\text{MPMM}(n, 1)$. The dimension n never changes.
Overhead	$O(n^2)$ per round (building D_0, S, X, Y , and the output), $O(n^2 \log n)$ in total.

Theorem 4.7 (MPMM reduces to APSP). $\text{MPMM}(n, W) \leq_{n^2} \text{APSP}(3n, W)$.

Construction. Build a layered DAG $X \rightarrow Y \rightarrow Z$ with $|X| = |Y| = |Z| = n$, edges $x_i \rightarrow y_k$ of weight $M[i, k]$ and $y_k \rightarrow z_j$ of weight $N[k, j]$ (entries $+\infty$ give no edge). Then $P[i, j] = \delta(x_i, z_j)$.

Calls	1 call to APSP.
Adaptivity	Non-adaptive.
Oracle instances	$\text{APSP}(3n, W)$.
Overhead	$O(n^2)$.

Together, [Theorems 4.6](#) and [4.7](#) give $\text{APSP} \equiv_3 \text{MPMM}$; this equivalence is classical [[FM71](#), [Mun71](#)].

4.4 The key reduction: MPMM reduces to APNWT-Det

This is the step that turns a function with n^2 integer outputs into a problem with n^2 one-bit outputs, using only $O(\log W)$ calls. The idea is a *simultaneous binary search*: every entry $P[i, j]$ is located by its own binary search, and one APNWT-Det call advances all n^2 searches by one step.

Definition 4.8 (Threshold instance). For $M, N \in (\mathbb{Z}_W^\infty)^{n \times n}$ and a threshold matrix $\Theta \in (\mathbb{Z} \cup \{\perp\})^{n \times n}$, let G_Θ be the tripartite instance on $A = \{a_i\}$, $B = \{b_k\}$, $C = \{c_j\}$ with

$$W_{AB}[a_i, b_k] = M[i, k], \quad W_{BC}[b_k, c_j] = N[k, j], \quad W_{AC}[a_i, c_j] = \begin{cases} -\Theta[i, j], & \Theta[i, j] \neq \perp \\ +\infty, & \Theta[i, j] = \perp \end{cases}$$

([Figure 1](#)).

Claim 4.9. Let $Y = \text{APNWT-Det}(G_\Theta)$ and $P = M * N$. For every (i, j) with $\Theta[i, j] \neq \perp$: $Y[i, j] = 1 \iff P[i, j] < \Theta[i, j]$.

Proof. $Y[i, j] = 1$ iff $\exists k : M[i, k] + N[k, j] - \Theta[i, j] < 0$ iff $\min_k (M[i, k] + N[k, j]) < \Theta[i, j]$. $\square$

Algorithm 1 MPMM via APNWT-Det (simultaneous binary search)

Require: $M, N \in (\mathbb{Z}_W^\infty)^{n \times n}$; oracle for APNWT-Det

Ensure: $P = M * N$

```

1:  $\Theta[i, j] \leftarrow 2W + 1$  for all  $(i, j)$ ;  $Y \leftarrow \text{APNWT-Det}(G_\Theta)$   $\triangleright$  finiteness round
2:  $\mathcal{F} \leftarrow \{(i, j) : Y[i, j] = 1\}$ ;  $P[i, j] \leftarrow +\infty$  for all  $(i, j) \notin \mathcal{F}$ 
3:  $L[i, j] \leftarrow -2W$ ,  $H[i, j] \leftarrow 2W + 1$  for all  $(i, j) \in \mathcal{F}$ 
4: for  $r = 1, \dots, R$  where  $R := \lceil \log(4W + 1) \rceil$  do
5:    $\Theta[i, j] \leftarrow \lfloor (L[i, j] + H[i, j])/2 \rfloor$  for  $(i, j) \in \mathcal{F}$ ;  $\Theta[i, j] \leftarrow \perp$  for  $(i, j) \notin \mathcal{F}$ 
6:    $Y \leftarrow \text{APNWT-Det}(G_\Theta)$ 
7:   for  $(i, j) \in \mathcal{F}$  do
8:     if  $Y[i, j] = 1$  then
9:        $H[i, j] \leftarrow \Theta[i, j]$   $\triangleright P[i, j] < \Theta[i, j]$ 
10:    else
11:       $L[i, j] \leftarrow \Theta[i, j]$   $\triangleright P[i, j] \geq \Theta[i, j]$ 
12:    end if
13:  end for
14: end for
15:  $P[i, j] \leftarrow L[i, j]$  for all  $(i, j) \in \mathcal{F}$ 
16: return  $P$ 

```

Theorem 4.10 (MPMM reduces to APNWT-Det; cf. [VWW18, Thm. 4.2]). *Algorithm 1 solves MPMM(n, W) with the profile below. Consequently, with $R = \lceil \log(4W + 1) \rceil$,*

$$T_{\text{MPMM}}(n, W) \leq (R + 1) T_{\text{APNWT-Det}}(n, 2W + 1) + O(n^2(R + 1)).$$

In particular $T_{\text{MPMM}}(n, W) = O(\log(W + 1) \cdot T_{\text{APNWT-Det}}(n, 2W + 1))$, as $T_{\text{APNWT-Det}}(n, \cdot) = \Omega(n^2)$ classically.

Calls	$1 + \lceil \log(4W + 1) \rceil = O(\log(W + 1))$ calls to APNWT-Det.
Adaptivity	Adaptive: $R + 1$ sequential rounds of one call each, since the thresholds of a round depend on all earlier answers. Within one call, all n^2 binary searches advance simultaneously.
Oracle instances	APNWT-Det($n, 2W + 1$): part size n ($3n$ vertices); the weights grow from W to $2W + 1$.
Overhead	$O(n^2)$ per round, $O(n^2 \log(W + 1))$ in total. The A – B and B – C matrices are the same in every call; only $W_{AC} = -\Theta$ changes.

Proof. Weights. M, N have entries in $\mathbb{Z}_W^\infty$. In the finiteness round the A – C weights are $-(2W + 1)$. In later rounds $L \geq -2W$ and $H \leq 2W + 1$ (both only move inward), and $H - L \geq 1$ (shown below), so $\Theta = \lfloor (L + H)/2 \rfloor \in [L, H - 1] \subseteq [-2W, 2W]$. Hence every oracle instance is an instance of APNWT-Det($n, 2W + 1$).

Finiteness round. Every finite entry of P lies in $\mathbb{Z}_{2W}$, so $P[i, j] < 2W + 1$ iff $P[i, j]$ is finite. By Claim 4.9, $\mathcal{F}$ is exactly the set of pairs with finite $P[i, j]$, and all other entries are correctly set to $+\infty$.

Invariant. For all $(i, j) \in \mathcal{F}$, after every round, $L[i, j] \leq P[i, j] < H[i, j]$. This holds initially since $P[i, j] \in \mathbb{Z}_{2W}$. Suppose it holds before a round and let $\Theta = \Theta[i, j]$. If $Y[i, j] = 1$ then $P[i, j] < \Theta$ by Claim 4.9, so setting $H \leftarrow \Theta$ preserves it. If $Y[i, j] = 0$ then $P[i, j] \geq \Theta$, so setting $L \leftarrow \Theta$ preserves it. In particular $H - L \geq 1$ at all times, because the integer $P[i, j]$ lies in $[L, H)$.

Progress. Let $d = H[i, j] - L[i, j]$ before a round. With $\Theta = L + \lfloor d/2 \rfloor$, the new gap is $\Theta - L = \lfloor d/2 \rfloor$ if H is updated, and $H - \Theta = \lceil d/2 \rceil$ if L is updated. Either way $d_{\text{new}} \leq \lceil d/2 \rceil$.

(If $d = 1$ then $\Theta = L$, and $Y[i, j] = 0$ because $P \geq L$; nothing changes, consistently with the invariant.) Initially $d = 4W + 1$. Since $\lceil \lceil x/2 \rceil / 2 \rceil = \lceil x/4 \rceil$ for integers x , after R rounds $d \leq \lceil (4W + 1)/2^R \rceil = 1$, because $2^R \geq 4W + 1$.

Output. After the last round $d = 1$, so the invariant gives $L[i, j] \leq P[i, j] < L[i, j] + 1$, i.e. $P[i, j] = L[i, j]$.

Cost. The algorithm makes $1 + R$ calls. Each round costs $O(n^2)$ to set Θ , write W_{AC} , and update L, H ; the matrices $W_{AB} = M$ and $W_{BC} = N$ are written once. $\square$

Remark 4.11 (Bounded-error oracles). Suppose the APNWT-Det oracle returns the *entire* correct matrix Y only with probability at least $2/3$, as quantum algorithms will. Replace each of the $R + 1$ calls by q independent calls and take the entrywise majority. If more than $q/2$ of the runs are fully correct, the majority is correct in every entry. Let X_r indicate that run r is fully correct and $S = \sum_{r=1}^q X_r$, so $\mathbb{E}[S] \geq 2q/3$. By Hoeffding's inequality,

$$\Pr[S \leq q/2] \leq \Pr[S - \mathbb{E}[S] \leq -q/6] \leq \exp(-2(q/6)^2/q) = e^{-q/18}.$$

Choosing $q = \lceil 18 \ln(3(R+1)) \rceil = O(\log \log W)$ makes each amplified call fail with probability at most $1/(3(R+1))$, and a union bound over the $R+1$ calls gives overall success probability at least $2/3$. This multiplies the number of calls by $O(\log \log W)$: $T_{\text{MPMM}}(n, W) = O(\log W \log \log W \cdot T_{\text{APNWT-Det}}(n, 2W + 1) + n^2 \log W \log \log W)$.

4.5 APNWT: finding reduces to detection

Finding reduces to detection in two steps. First, APNWT-Find reduces to a *single* MPMM call on a re-encoded instance (Lemma 4.12). Second, that MPMM call is answered by the key reduction Theorem 4.10 (Corollary 4.13). The weight bound changes along the way: detection is queried with weights up to $2n(W + 1) + 1$, not W .

Lemma 4.12 (APNWT-Find reduces to one MPMM call). $\text{APNWT-Find}(n, W) \leq \text{MPMM}(n, n(W + 1))$ with the profile below.

Construction. Index $B = \{b_0, \dots, b_{n-1}\}$ and form the MPMM instance

$$M'[i, k] := n \cdot W_{AB}[a_i, b_k] + k, \quad N'[k, j] := n \cdot W_{BC}[b_k, c_j] \quad (\infty \text{ stays } \infty),$$

whose entries lie in $\mathbb{Z}_{n(W+1)}^\infty$. Query the oracle once to obtain $P' = M' * N'$. For finite $P'[i, j]$, write $P'[i, j] = nq + r$ with $0 \leq r < n$ (Euclidean division). Then $q = P[i, j] := \min_k (W_{AB}[a_i, b_k] + W_{BC}[b_k, c_j])$, and r is the smallest index attaining this minimum (Lemma A.3). Output

$$F[a_i, c_j] = \begin{cases} b_r, & P'[i, j] < \infty \text{ and } q + W_{AC}[a_i, c_j] < 0, \\ \perp, & \text{otherwise.} \end{cases}$$

This is correct because (a_i, c_j) has a witness iff $P[i, j] + W_{AC}[a_i, c_j] < 0$ (Remark 3.20), and in that case every minimiser, in particular b_r , is a witness.

Calls	1 call to MPMM.
Adaptivity	Non-adaptive.
Oracle instances	$\text{MPMM}(n, n(W + 1))$: the dimension equals the part size n ; the weights grow by a factor of about n because of the lexicographic encoding.
Overhead	$O(n^2)$ for encoding M', N' and decoding P' .

Corollary 4.13 (APNWT: finding from detection). *Composing Lemma 4.12 with Theorem 4.10 (applied with weight bound $n(W + 1)$) gives $\text{APNWT-Find}(n, W) \leq \text{APNWT-Det}$ with the profile below. With $R'' = \lceil \log(4n(W + 1) + 1) \rceil$,*

$$T_{\text{APNWT-Find}}(n, W) \leq (R'' + 1) T_{\text{APNWT-Det}}(n, 2n(W + 1) + 1) + O(n^2(R'' + 1)).$$

Algorithm 2 APNWT-Find via NWT-Find

Require: tripartite instance (W_{AB}, W_{BC}, W_{AC}) with parameters (n, W) ; oracle for NWT-Find

Ensure: $F \in (B \cup \{\perp\})^{A \times C}$

```
1:  $s \leftarrow \lceil n^{1/3} \rceil$ ,  $g \leftarrow \lceil n/s \rceil$ ; partition  $A = \bigsqcup_{p=1}^g A_p$ ,  $B = \bigsqcup_{q=1}^g B_q$ ,  $C = \bigsqcup_{r=1}^g C_r$  into blocks of
   size at most  $s$ 
2:  $F[a, c] \leftarrow \perp$  for all  $(a, c)$ ;  $\widetilde{W}_{AC} \leftarrow W_{AC}$   $\triangleright$  working copy; entries only ever set to  $+\infty$ 
3: for each  $(p, q, r) \in [g]^3$  do
4:   loop
5:      $\tau \leftarrow \text{NWT-Find on } (W_{AB}|_{A_p \times B_q}, W_{BC}|_{B_q \times C_r}, \widetilde{W}_{AC}|_{A_p \times C_r})$ , padded to size  $s$ 
6:     if  $\tau = \perp$  then break
7:     end if
8:      $(a, b, c) \leftarrow \tau$ ;  $F[a, c] \leftarrow b$ ;  $\widetilde{W}_{AC}[a, c] \leftarrow +\infty$   $\triangleright$  delete edge  $a \rightarrow c$ 
9:   end loop
10: end for
11: return  $F$ 
```

Calls	$1 + \lceil \log(4n(W+1)+1) \rceil = O(\log(nW))$ calls to APNWT-Det.
Adaptivity	Adaptive, with $O(\log(nW))$ sequential rounds (those of Algorithm 1 run on M', N').
Oracle instances	APNWT-Det($n, 2n(W+1)+1$): the part size is unchanged; the weights grow from W to $2n(W+1)+1$.
Overhead	$O(n^2)$ for encoding and decoding, plus $O(n^2 \log(nW))$ inside Algorithm 1 .

Together with [Observation 3.19](#), detection and finding for APNWT have the same complexity up to an $O(\log(nW))$ factor and a change of the weight bound from W to $2n(W+1)+1$. Under [Assumption 3.4](#) this bound is still $n^{O(1)}$, i.e. $\Theta(\log n)$ bits.

Remark 4.14 (Why the halving argument of [Lemma 4.2](#) does not transfer directly). Halving B tells each pair (a, c) which half of B contains a witness, but different pairs may need *different* halves. One APNWT-Det call cannot restrict the witness range per pair, since that restriction depends jointly on (a, c) and b and is not expressible through the three edge weights. Recursing on each half separately would cost $\Theta(n)$ calls in total. The lexicographic encoding above avoids this: it makes the minimiser, which is always a witness when one exists, readable from the *value* of a min-plus product, and values can be searched simultaneously.

4.6 APNWT reduces to NWT-Find

Here the n^2 outputs of APNWT are produced by many NWT calls on *small* sub-instances. Each call either certifies that a block contains no further triangles, or finds a triangle, whose A - C edge is then deleted. The reduction directly solves the finding version.

Theorem 4.15 (APNWT reduces to NWT-Find; cf. [\[VWW18, Lem. 4.2\]](#)). *Algorithm 2 solves APNWT-Find(n, W), and hence APNWT-Det(n, W), with the profile below. Consequently, with $s = \lceil n^{1/3} \rceil$,*

$$T_{\text{APNWT-Find}}(n, W) \leq 9n^2 T_{\text{NWT-Find}}(s, W) + O(n^2 s^2).$$

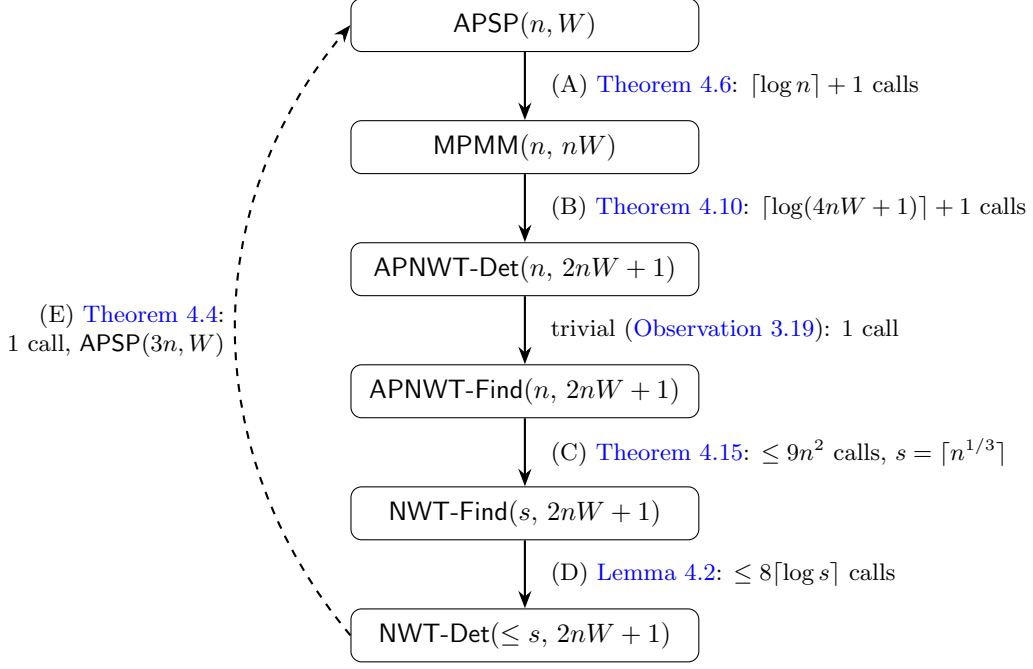


Figure 2: The reduction pathway. Each box shows the parameters of the oracle instances issued by the arrow entering it. Calls are counted per query of the preceding problem.

Calls	At most $g^3 + n^2 \leq 9n^2$ calls to NWT-Find: each block triple ends with exactly one $\perp$ -answer ($g^3 \leq (n^{2/3} + 1)^3 \leq 8n^2$ of them), and each non- $\perp$ answer deletes a distinct finite entry of W_{AC} (at most n^2 of them).
Adaptivity	Adaptive: later sub-instances depend on the edges deleted so far.
Oracle instances	NWT-Find(s, W) with $s = \lceil n^{1/3} \rceil$. The part size drops from n to about $n^{1/3}$ and the weights are unchanged. Relative to the new size they remain polynomial: $W \leq n^c \leq s^{3c}$, so $\Theta(\log n)$ -bit weights become $\Theta(\log s)$ -bit weights.
Overhead	$O(s^2)$ per call to write the sub-instance, so $O(n^2 s^2) = O(n^{8/3})$ in total; $O(n^2)$ with implicit sub-instances (Remark 2.4).

Remark 4.16. Combining Theorem 4.15 with Lemma 4.2: if NWT-Det($\cdot, W$) runs in regular time $T(\cdot)$, then $T_{\text{APNWT-Find}}(n, W) \leq 9n^2(16T(s) + O(s^2)) + O(n^2 s^2) = O(n^2 T(\lceil n^{1/3} \rceil))$. The number of NWT-Det calls is at most $9n^2 \cdot 8\lceil \log s \rceil = O(n^2 \log n)$.

5 Main Result: from APSP to NWT and back

Table 1 collects the profiles of the arrows in Figure 2. Every arrow is a Turing reduction.

Evolution of the parameters. Start from APSP(n, W) with $W \leq n^c$ (Assumption 3.4). Then:

$$(n, n^c) \xrightarrow{(A)} (n, n^{c+1}) \xrightarrow{(B)} (n, 2n^{c+1} + 1) \xrightarrow{(C)} (s, 2n^{c+1} + 1) \xrightarrow{(D)} (\leq s, 2n^{c+1} + 1),$$

where $s = \lceil n^{1/3} \rceil$. The size shrinks only in (C) (and in the halving of (D)); the weights grow only in (A) (by a factor n , because walks of up to n edges are summed) and (B) (by a factor 2, plus 1 for the finiteness threshold).

Remark 5.1 (The weight regime is preserved). For $n > 8$ we have $s \geq 3$ and $n \leq s^3$, so $2n^{c+1} + 1 \leq 3n^{c+1} \leq 3s^{3(c+1)} \leq s^{3c+4}$. Thus the NWT oracle is only ever queried on instances

Arrow	Calls/query	Adapt.	Oracle instance	Overhead
(A) APSP $\rightarrow$ MPMM	$\lceil \log n \rceil + 1$	yes	MPMM(n, nW)	$O(n^2 \log n)$
(B) MPMM $\rightarrow$ APNWT-Det	$\lceil \log(4W' + 1) \rceil + 1$	yes	APNWT-Det(n, W'')	$O(n^2 \log W')$
APNWT-Det $\rightarrow$ APNWT-Find	1	no	same	$O(n^2)$
(C) APNWT-Find $\rightarrow$ NWT-Find	$\leq 9n^2$	yes	NWT-Find(s, W'')	$O(n^{8/3})$
(D) NWT-Find $\rightarrow$ NWT-Det	$\leq 8 \lceil \log s \rceil$	yes	NWT-Det($\lceil s/2^i \rceil, W''$)	$O(s^2)$
(E) NWT-Det $\rightarrow$ APSP	1	no	APSP($3n, W$)	$O(n^2)$

Table 1: Profiles along the pathway. Here $W' = nW$, $W'' = 2W' + 1 = 2nW + 1$, $s = \lceil n^{1/3} \rceil$ is the part size of the NWT-Find queries, and i in (D) is the round index.

with $\Theta(\log s)$ -bit weights, relative to its own size s , and [Assumption 3.4](#) is respected at every level of the chain.

Theorem 5.2 (APSP from NWT, quantitative). *Let $T : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{R}_{>0}$ be such that NWT-Det(m, W') can be solved in time $T(m, W')$ for all m, W' , and $m \mapsto T(m, W')$ is regular ([Assumption 4.1](#)) for every fixed W' . Then for all $n \geq 2$ and $W \geq 1$, with $s = \lceil n^{1/3} \rceil$,*

$$T_{\text{APSP}}(n, W) = O\left(\log n \cdot \log(nW) \cdot n^2 \cdot T(s, 2nW + 1)\right).$$

The resulting algorithm makes $O(n^2 \log^2 n \log(nW))$ calls to NWT-Det, organised in adaptive rounds, all on instances with part size at most s and weights in $\mathbb{Z}_{2nW+1}$.

Proof. Compose the reductions (A)–(D) using (1) and [Remark 2.3](#). Every query below is an instance with weights bounded by $W'' := 2nW + 1$; queries with smaller weights, such as those from the final MPMM($n, 1$) call in (A), are also instances with weight bound W'' . Let $t = \lceil \log n \rceil$ and $R' = \lceil \log(4nW + 1) \rceil$, and write $T(\cdot)$ for $T(\cdot, W'')$.

- (D) [Lemma 4.2](#): $T_{\text{NWT-Find}}(s, W'') \leq 16T(s) + O(s^2)$.
- (C) [Theorem 4.15](#) and [Observation 3.19](#): $T_{\text{APNWT-Det}}(n, W'') \leq 9n^2(16T(s) + O(s^2)) + O(n^2 s^2)$.
- (B) [Theorem 4.10](#) with $W' = nW$: $T_{\text{MPMM}}(n, nW) \leq (R' + 1)T_{\text{APNWT-Det}}(n, W'') + O(n^2(R' + 1))$.
- (A) [Theorem 4.6](#): $T_{\text{APSP}}(n, W) \leq (t + 1)T_{\text{MPMM}}(n, nW) + O(n^2(t + 1))$.

Substituting, and using $T(s) \geq s^2$ (regularity) to absorb all $O(s^2)$ and $O(n^2)$ terms,

$$T_{\text{APSP}}(n, W) \leq (t + 1)(R' + 1) \cdot O(n^2 T(s)) = O(\log n \cdot \log(nW) \cdot n^2 T(s)).$$

The number of NWT-Det calls is at most $(t+1)(R'+1) \cdot 9n^2 \cdot 8 \lceil \log s \rceil = O(n^2 \log^2 n \log(nW))$. $\square$

Corollary 5.3 (Subcubic equivalence). *The following problems are pairwise subcubic equivalent: APSP, MPMM, APNWT-Det, APNWT-Find, NWT-Det and NWT-Find, where NWT may be taken in any of its transitive/circular and tripartite/general variants. Quantitatively, if NWT-Det runs in time $O(n^{3-\delta} \log^d(W + 1))$, then APSP runs in time $O(n^{3-\delta/3} \log^{d+2}(nW))$; conversely, an $O(n^{3-\delta} \log^d(W + 1))$ algorithm for APSP gives an $O(n^{3-\delta} \log^d(W + 1))$ algorithm for NWT-Det.*

Proof. NWT $\Rightarrow$ APSP. Assume $T_{\text{NWT-Det}}(m, W') \leq Km^{3-\delta} \log^d(W' + 1)$, and without loss of generality $\delta \leq 1$ and $K \geq 1$. For fixed W' , the function $T(m) := Km^{3-\delta} \log^d(W' + 1)$ is regular: $T(m)/m = Km^{2-\delta} \log^d(W' + 1)$ is nondecreasing, and $T(m) \geq m^2$. By [Theorem 5.2](#), and since $s^{3-\delta} \leq (2n^{1/3})^{3-\delta} \leq 8n^{1-\delta/3}$ and $\log(2nW + 2) = O(\log(nW))$ for $n \geq 2$,

$$T_{\text{APSP}}(n, W) = O(\log n \log(nW) \cdot n^2 \cdot n^{1-\delta/3} \log^d(nW)) = O(n^{3-\delta/3} \log^{d+2}(nW)),$$

which is truly subcubic (with $\delta/3$ in place of δ ; the factor $\log^{d+2}(nW) \leq 2^{d+2}(\log^{d+2} n + \log^{d+2}(W + 1))$ is absorbed by lowering the exponent slightly).

APSP $\Rightarrow$ NWT. By [Theorem 4.4](#), $T_{\text{NWT-Det}}(n, W) \leq T_{\text{APSP}}(3n, W) + O(n^2)$.

All the other problems. Consider the cycle

$$\text{APSP} \leq \text{MPMM} \leq \text{APNWT-Det} \leq \text{APNWT-Find} \leq \text{NWT-Find} \leq \text{NWT-Det} \leq \text{APSP},$$

given by [Theorems 4.6](#) and [4.10](#), [Observation 3.19](#), [Theorem 4.15](#), [Lemma 4.2](#) and [Theorem 4.4](#). Every arrow preserves truly subcubic running times: (A) and (B) make $O(\log(nW))$ calls on instances of the same dimension with polynomially related weights; (C) turns an $O(m^{3-\delta} \text{polylog})$ bound into $O(n^2 \cdot n^{1-\delta/3} \text{polylog})$; (D) and the trivial arrows lose only constant factors; and [Theorem 4.4](#) multiplies the size by 3. Hence a truly subcubic algorithm for any one problem in the cycle yields one for all of them. The variants of NWT are covered by [Propositions 3.11](#) and [3.12](#), which are $\leq n^2$ reductions with size at most tripled. $\square$

Remark 5.4. (i) *Exponent loss.* An $O(n^{3-\delta})$ algorithm for NWT yields $\tilde{O}(n^{3-\delta/3})$ for APSP; for example, $n^{2.9}$ becomes about $n^{2.967}$. The loss comes from step (C), which issues $\Theta(n^2)$ calls on instances of size $n^{1/3}$. Whether $O(n^{3-\delta})$ for NWT implies $O(n^{3-\delta})$ for APSP is open [[VWW18](#), Sec. 9].

- (ii) *Where adaptivity is essential.* All forward arrows (A)–(D) are adaptive. In (A) and (B) the depth is only logarithmic ($O(\log n)$ and $O(\log(nW))$ rounds); in (C) it can be $\Theta(n^2)$. This will matter for the quantum analysis.
- (iii) *Decision vs. function.* NWT-Det returns one bit, whereas APSP returns n^2 numbers of $\Theta(\log n)$ bits. The chain bridges this gap in two places. Step (B) extracts $\Theta(\log W)$ bits per entry through a simultaneous binary search. Step (C) extracts n^2 output bits from one-bit answers on $n^{1/3}$ -size instances, using the fact that each positive answer “pays for” one output entry.

A Deferred Proofs

A.1 Proof of [Proposition 3.11](#) (orientation)

Circular to transitive. Given a circular instance (W_{AB}, W_{BC}, W_{CA}) , output the transitive instance (W_{AB}, W_{BC}, W_{AC}) with $W_{AC} := W_{CA}^\top$, that is, $W_{AC}[a, c] = W_{CA}[c, a]$. For every (a, b, c) the circular weight $W_{AB}[a, b] + W_{BC}[b, c] + W_{CA}[c, a]$ equals the transitive weight $W_{AB}[a, b] + W_{BC}[b, c] + W_{AC}[a, c]$. Hence the two instances have exactly the same negative triangles (as triples), and a detection answer or a found triple can be returned unchanged. Transposition takes $O(n^2)$ time, the parameters (n, W) are unchanged, and one non-adaptive call is made. *Transitive to circular* is identical, with $W_{CA} := W_{AC}^\top$. $\square$

A.2 Proof of [Proposition 3.12](#) (tripartite vs. general)

General to tripartite (both orientations). Given W_G on V , $|V| = n$, let A, B, C be three copies of V and set $W_{AB} = W_{BC} = W_G$ together with $W_{AC} = W_G$ (transitive) or $W_{CA} = W_G$ (circular). A triple (u_A, v_B, x_C) has tripartite weight $W_G[u, v] + W_G[v, x] + W_G[u, x]$ (respectively

$+W_G[x, u]$), which is exactly the general weight of (u, v, x) . So negative triangles correspond bijectively, and distinctness is automatic (Definition 3.10). The parameters are (n, W) , with $O(n^2)$ time and one call.

Tripartite to general, transitive. Let $V = A \sqcup B \sqcup C$ ($3n$ vertices) and let W_G be the block matrix with W_{AB}, W_{BC}, W_{AC} in blocks $A \times B, B \times C, A \times C$, and $+\infty$ elsewhere. Every tripartite negative triangle is a general one with the same weight. Conversely, let (u, v, x) be a general negative triangle, so $W_G[u, v], W_G[v, x], W_G[u, x]$ are finite. Finite entries have rows in $A \cup B$ and columns in $B \cup C$. Since v is both a column index (of $W_G[u, v]$) and a row index (of $W_G[v, x]$), $v \in B$. The only finite block with columns in B is $A \times B$, so $u \in A$; the only finite block with rows in B is $B \times C$, so $x \in C$. Hence $(u, v, x) \in A \times B \times C$ is a tripartite negative triangle.

Tripartite to general, circular. Use the blocks W_{AB}, W_{BC}, W_{CA} in $A \times B, B \times C, C \times A$. Every finite entry leads from a part to the cyclically next part ($A \rightarrow B \rightarrow C \rightarrow A$). A general circular negative triangle $u \rightarrow v \rightarrow x \rightarrow u$ therefore visits three consecutive parts in cyclic order, and its rotation starting in A is a tripartite circular negative triangle of the same weight. The converse is immediate. The parameters are $(3n, W)$, with $O(n^2)$ time and one call. $\square$

A.3 Structure of walks and shortest paths

Lemma A.1. *Let G be a weighted digraph on n vertices without self-loops.*

- (a) *Every closed walk of negative weight contains (as a sub-sequence of its edges) a negative-weight cycle, all of whose vertices lie on the walk.*
- (b) *$\delta(u, v) = -\infty$ iff there exists a vertex z on a negative-weight cycle with $u \rightsquigarrow z \rightsquigarrow v$.*
- (c) *If $\delta(u, v) \notin \{\pm\infty\}$, then $\delta(u, v)$ is the weight of a path from u to v , which has at most $n - 1$ edges.*

Proof. (a) A closed walk decomposes into cycles whose weights sum to the weight of the walk (repeatedly cut out the segment between the first repeated vertex and its earlier occurrence). If the total is negative, some cycle in the decomposition is negative.

(b) ($\Leftarrow$) The walk $u \rightsquigarrow z$, then around the cycle k times, then $z \rightsquigarrow v$ has weight tending to $-\infty$ as $k \rightarrow \infty$. ($\Rightarrow$) Suppose no vertex z with $u \rightsquigarrow z \rightsquigarrow v$ lies on a negative cycle. Let p be any walk from u to v . Every vertex of p satisfies $u \rightsquigarrow z \rightsquigarrow v$. Cutting cycles out of p (as in (a)) removes cycles on such vertices, which are non-negative, so the result is a path p' from u to v with $w(p') \leq w(p)$. Hence $\delta(u, v)$ is a minimum over the finitely many paths from u to v , so it is not $-\infty$.

(c) This follows from the argument for ($\Rightarrow$) in (b): the infimum is attained by a path, and a path has at most $n - 1$ edges. $\square$

A.4 Proof of Theorem 4.4 (NWT to APSP)

In H every edge goes from A to B or from B to C , so H is acyclic and every walk from $a \in A$ to $c \in C$ is of the form $a \rightarrow b \rightarrow c$. Hence $\delta_H(a, c) = \min_b (W_{AB}[a, b] + W_{BC}[b, c])$, which is $+\infty$ if no such walk exists, and $\delta_H(a, c)$ is never $-\infty$. Therefore

$$\exists b : W_{AB}[a, b] + W_{BC}[b, c] + W_{AC}[a, c] < 0 \iff \delta_H(a, c) + W_{AC}[a, c] < 0.$$

Taking the disjunction over all (a, c) gives NWT-Det, and keeping the individual bits gives APNWT-Det (Remark 4.5). The graph H has $3n$ vertices and weights in $\mathbb{Z}_W$. Building H and scanning $A \times C$ take $O(n^2)$ time. $\square$

A.5 Proof of Theorem 4.6 (APSP to MPMM)

Lemma A.2 (Repeated squaring). *For $s \geq 0$, $D_s[u, v]$ equals the minimum weight of a walk from u to v with at most 2^s edges (and $+\infty$ if there is none).*

Proof. Induction on s . For $s = 0$: walks with at most one edge are the empty walk (only when $u = v$, weight 0) and single edges. Since there are no self-loops, $D_0[u, u] = 0$ and $D_0[u, v] = W_G[u, v]$ for $u \neq v$, as claimed. For $s \geq 1$: $D_s[u, v] = \min_x (D_{s-1}[u, x] + D_{s-1}[x, v])$. Concatenating a walk $u \rightsquigarrow x$ and a walk $x \rightsquigarrow v$, each with at most 2^{s-1} edges, gives a walk with at most 2^s edges, so the right-hand side is at least the claimed minimum. Conversely, a walk with $\ell \leq 2^s$ edges splits at its $\min(\ell, 2^{s-1})$ -th vertex x into two walks with at most 2^{s-1} edges each, so the right-hand side is at most the claimed minimum. $\square$

Oracle instances. By Lemma A.2, every finite entry of D_{s-1} is the weight of a walk with at most 2^{s-1} edges, hence lies in $\mathbb{Z}_{2^{s-1}W}$. The s -th call is therefore an instance of MPMM($n, 2^{s-1}W$). Since $t = \lceil \log n \rceil$ gives $2^{t-1} < n$, all squarings are instances of MPMM(n, nW). The matrices X, Y have entries in $\{0, +\infty\}$, giving MPMM($n, 1$).

Correctness. Write $D = D_t$, where $2^t \geq n$.

- (i) *Reachability.* $D[u, v] < \infty$ iff $u \rightsquigarrow v$: if $u \rightsquigarrow v$ there is a path with at most $n - 1 \leq 2^t$ edges, and $D[u, u] = 0$.
- (ii) *S is correct.* If z lies on a negative cycle, that cycle has at most $n \leq 2^t$ edges, so $D[z, z] < 0$ and $z \in S$. Conversely, if $D[z, z] < 0$ there is a negative closed walk through z . By Lemma A.1(a) it contains a negative cycle $\mathcal{C}$, and every vertex y of $\mathcal{C}$ satisfies $z \rightsquigarrow y \rightsquigarrow z$.
- (iii) *$-\infty$ entries.* $Z[u, v] = 0$ iff there is $z \in S$ with $D[u, z] < \infty$ and $D[z, v] < \infty$, i.e. with $u \rightsquigarrow z \rightsquigarrow v$ by (i). If $\delta(u, v) = -\infty$, Lemma A.1(b) gives a z on a negative cycle with $u \rightsquigarrow z \rightsquigarrow v$, and $z \in S$ by (ii). Conversely, if $z \in S$ and $u \rightsquigarrow z \rightsquigarrow v$, take $\mathcal{C}$ and y as in (ii). Then $u \rightsquigarrow z \rightsquigarrow y \rightsquigarrow z \rightsquigarrow v$ with y on a negative cycle, so $\delta(u, v) = -\infty$. (If G has a negative self-loop at z , add z to S explicitly; the same argument applies.)
- (iv) *Other entries.* If $\delta(u, v) > -\infty$, then every walk has weight at least $\delta(u, v)$, so $D[u, v] \geq \delta(u, v)$. If moreover $\delta(u, v) < \infty$, it is attained by a path with at most $n - 1 \leq 2^t$ edges (Lemma A.1(c)), so $D[u, v] \leq \delta(u, v)$. If $\delta(u, v) = +\infty$ then $D[u, v] = +\infty$ by (i).

Cost. There are $t + 1$ calls, and $O(n^2)$ work per round. $\square$

A.6 Proof of Theorem 4.7 (MPMM to APSP)

The graph is a DAG with layers $X \rightarrow Y \rightarrow Z$, so every walk from x_i to z_j is $x_i \rightarrow y_k \rightarrow z_j$ for some k with $M[i, k], N[k, j] < \infty$. Hence $\delta(x_i, z_j) = \min_k (M[i, k] + N[k, j]) = P[i, j]$, including the value $+\infty$ when no such k exists. There are $3n$ vertices, weights in $\mathbb{Z}_W$, and $O(n^2)$ time. $\square$

A.7 Proof of Lemma 4.12 (APNWT-Find to MPMM)

Lemma A.3 (Lexicographic encoding). *Let $v_0, \dots, v_{n-1} \in \mathbb{Z} \cup \{+\infty\}$, not all $+\infty$. Let $v^* = \min_k v_k$ and $k^* = \min\{k : v_k = v^*\}$. Then $\min_k (nv_k + k) = nv^* + k^*$, and Euclidean division of this value by n has quotient v^* and remainder k^* .*

Proof. If $v_k = v^*$ then $nv_k + k \geq nv^* + k^*$ by the choice of k^* . If $v_k \geq v^* + 1$ (including $v_k = \infty$) then $nv_k + k \geq nv^* + n > nv^* + k^*$, since $k^* \leq n - 1$. Finally, $0 \leq k^* \leq n - 1$ makes (v^*, k^*) the quotient and remainder. $\square$

Correctness. Fix (i, j) and let $v_k = W_{AB}[a_i, b_k] + W_{BC}[b_k, c_j]$. Then $M'[i, k] + N'[k, j] = nv_k + k$, so $P'[i, j] = \min_k(nv_k + k)$. If all $v_k = \infty$, then $P'[i, j] = \infty$ and (a_i, c_j) has no witness, so the output $\perp$ is correct. Otherwise Lemma A.3 gives $q = v^* = P[i, j]$ and $r = k^*$. By Remark 3.20, (a_i, c_j) has a witness iff $v^* + W_{AC}[a_i, c_j] < 0$, and in that case $v_r + W_{AC}[a_i, c_j] = v^* + W_{AC}[a_i, c_j] < 0$, so b_r is a witness.

Parameters. $|M'[i, k]| \leq nW + n - 1 \leq n(W + 1)$ and $|N'[k, j]| \leq nW$, so the single query is an instance of $\text{MPMM}(n, n(W + 1))$. Encoding and decoding take $O(n^2)$ time (one multiplication, addition, or Euclidean division by n per entry). This proves Lemma 4.12.

Corollary 4.13. Answer the query by Theorem 4.10 with weight bound $W' = n(W + 1)$. This takes $1 + \lceil \log(4W' + 1) \rceil$ adaptive calls to $\text{APNWT-Det}(n, 2W' + 1) = \text{APNWT-Det}(n, 2n(W + 1) + 1)$ and $O(n^2 \log(nW))$ overhead; the cost bound follows from (1) and Remark 2.3. $\square$

A.8 Proof of Theorem 4.15 (APNWT to NWT-Find)

Invariant. Every entry of $\widetilde{W}_{AC}$ is either the original entry of W_{AC} or $+\infty$, and $\widetilde{W}_{AC}[a, c] = +\infty$ was set exactly when $F[a, c]$ was assigned. The matrices W_{AB}, W_{BC} are never modified.

Soundness. Suppose $F[a, c] = b$ is assigned. Then (a, b, c) was returned by NWT-Find on a sub-instance in which $\widetilde{W}_{AC}[a, c]$ was still finite, hence equal to $W_{AC}[a, c]$. So $W_{AB}[a, b] + W_{BC}[b, c] + W_{AC}[a, c] < 0$, and b is a witness in G . Immediately afterwards $\widetilde{W}_{AC}[a, c] = +\infty$, so no later triangle uses (a, c) , and $F[a, c]$ is assigned at most once.

Completeness. Suppose (a, c) has a witness b in G but $F[a, c] = \perp$ at the end. Then $\widetilde{W}_{AC}[a, c] = W_{AC}[a, c]$ throughout. Let (p, q, r) be the block triple with $a \in A_p, b \in B_q, c \in C_r$. Its loop ended with NWT-Find returning $\perp$ on a sub-instance containing (a, b, c) with its original weights. But (a, b, c) is a negative triangle there, a contradiction.

Calls. We have $g = \lceil n/s \rceil \leq n/s + 1 \leq n^{2/3} + 1$, so $g^3 \leq (n^{2/3} + 1)^3 = 8n^2$. Each block triple's loop makes exactly one call answered by $\perp$. Every other call deletes a finite entry of $\widetilde{W}_{AC}$, and there are at most n^2 of these. Hence there are at most $9n^2$ calls, each on an instance of part size at most s (padded to exactly s by Remark 2.1) with weights in $\mathbb{Z}_W$. Writing a sub-instance costs $O(s^2)$, so the overhead is $O(n^2 s^2)$, where $s^2 \leq (n^{1/3} + 1)^2 \leq 4n^{2/3}$. $\square$

References

- [AGM97] N. Alon, Z. Galil, and O. Margalit. On the exponent of the all pairs shortest path problem. *J. Comput. Syst. Sci.*, 54(2):255–262, 1997.
- [FM71] M. J. Fischer and A. R. Meyer. Boolean matrix multiplication and transitive closure. In *Proc. 12th Annual Symposium on Switching and Automata Theory (SWAT, later FOCS)*, pages 129–131, 1971.
- [Mun71] J. I. Munro. Efficient determination of the transitive closure of a directed graph. *Inf. Process. Lett.*, 1(2):56–58, 1971.
- [VWW18] V. Vassilevska Williams and R. R. Williams. Subcubic equivalences between path, matrix, and triangle problems. *J. ACM*, 65(5):27:1–27:38, 2018.
- [Wil14] R. Williams. Faster all-pairs shortest paths via circuit complexity. In *Proc. 46th STOC*, pages 664–673, 2014.